ARCHITECTURE REFERENCE

Security & RBAC Implementation

How data-access control works in the InspectAI NL2SQL prototype, end to end — for engineers extending, auditing, or onboarding onto this system.

Applies to	InspectAI NL2SQL prototype (inspection domain)
Covers	Authentication, permission model, SQL-level scope enforcement, write blocking, disclosure controls, audit logging
Source of truth	`core/auth.py`, `auth_routes.py`, `core/rbac.py`, `core/validator.py`, `agent/orchestrator.py`, `fastapi_app.py`, `core/logging_config.py`
Last verified against source	2026-07-10

1 Design principle: two independent enforcement layers
2 Request lifecycle overview
3 Authentication
4 Permission fetch & whitelist
5 RBAC scope resolution — `build_scope_filter`
6 SQL-level enforcement — `enforce_scope`
7 How to add a new protected table
8 Write-operation blocking
9 Information-disclosure redaction
10 Security audit logging
11 Design limitations & things to revisit

1. Design principle: two independent enforcement layers

Data-access control is enforced in two separate places, and this separation is the central fact to understand before touching anything in this system:

1. **The LLM is told what it's allowed to see**, via a line injected into its system prompt at query time.
2. **The SQL it produces is rewritten to enforce that restriction**, regardless of what it actually generated, by a deterministic, non-LLM code path (`core/validator.py::enforce_scope`).

> **WHY BOTH LAYERS EXIST**
>
> Prompt instructions are advisory — a model can misread a question, get talked out of a rule by a cleverly worded request, or simply behave inconsistently across runs of the same question. Every restriction that actually matters for data safety is therefore re-applied at the SQL layer by parsing the generated SQL into a real syntax tree and injecting a WHERE-clause predicate into it directly. **Layer 2 does not ask the model's permission and does not trust anything layer 1 already did.** Any future feature that reads or writes inspection data must go through layer 2, or it isn't actually protected — a correct prompt is not sufficient on its own.

2. Request lifecycle overview

#	Layer	File	Question it answers
1	Authentication	`core/auth.py`, `auth_routes.py`	Is this a real, currently-valid session?
2	Permission fetch & whitelist	`auth_routes.py`, `core/rbac.py`	What is this person allowed to see, and which of those permissions are relevant to the SQL layer at all?
3	RBAC scope resolution	`core/rbac.py::build_scope_filter`	Translate permissions into one concrete outcome: unrestricted, a specific WHERE-clause predicate, or a hard block.
4	SQL-level enforcement	`core/validator.py::enforce_scope`	Rewrite whatever SQL the model produced so that predicate is actually present, regardless of what the model wrote.
5	Write blocking	`core/validator.py` (`_DML_RE`)	Refuse to execute anything that isn't a SELECT, independent of role.

Two more controls sit outside this pipeline, applied to the response rather than the query:

Control	File	Question it answers
Disclosure redaction	`fastapi_app.py::_redact_security_internals`	Did the model's own explanation text leak the enforcement mechanism it's instructed not to talk about?
Security audit logging	`core/logging_config.py`, `[SCOPE]`-tagged lines	Is there a durable, greppable record of every scope decision made?

3. Authentication

Login is a proxy, not a local credential check. `POST /api/login` forwards the username/password to the real identity server via an OAuth2 password grant — client credentials never reach the browser, and the authentication decision itself is made by that external server.

`auth_routes.py — login()`

```
payload = {
    "grant_type":    "password",
    "client_id":     client_id,
    "client_secret": client_secret,
    "username":      req.username,
    "password":      req.password,
}
resp = await client.post(oauth_url, data=payload)   # real auth happens here
if resp.status_code == 200:
    access_token = resp.json()["access_token"]
    claims = _decode_jwt_payload(access_token)        # sub extraction only
    sub = str(claims.get("sub", "")).strip()
    if not re.match(r'^\d+$', sub) or sub == "0":
        raise HTTPException(401, "Invalid token: sub must be a positive integer")
```

> **WHY THE JWT SIGNATURE ISN'T RE-VERIFIED**
>
> `_decode_jwt_payload` decodes the token's payload without checking its signature. This is safe here specifically because the token was just issued by the trusted OAuth server in the same request/response — it is never re-validated cryptographically on subsequent calls. From that point on, every protected route checks the token only against this server's own in-memory `_valid_tokens` store, which is populated exclusively by a successful OAuth exchange. **That store, not the JWT signature, is the real authorization boundary.** If this system is ever changed to accept tokens it didn't itself just receive from the OAuth server (e.g. a token forwarded from another service), signature verification would need to be added.

Two additional checks run before a session is granted, independent of what the OAuth server said:

- **Sub format validation** — the decoded `sub` claim must be a positive integer; anything else is rejected before it can enter the token store.
- **Local sync check** — a DB lookup (`SELECT id_fk FROM users WHERE id_fk = sub`) confirms this user exists and is synced locally. A user who authenticates successfully upstream but has `id_fk = -1` (unsynced) is rejected with 403.

Every protected route depends on the `require_auth` FastAPI dependency, which checks the bearer token against the in-memory store and its expiry — nothing downstream ever re-inspects the JWT itself.

4. Permission fetch & whitelist

After login, the client calls `GET /api/me/permissions`, which resolves the Drupal integer UID to the internal UUID and fetches the user's full permission list from a separate internal Permissions API. That list commonly contains 200+ entries covering the entire application, most of which have nothing to do with data visibility.

> **THE WHITELIST BOUNDARY**
>
> Only permissions present in `core.rbac.DATA_VISIBILITY_PERMS` are ever forwarded past this point. This is a hard filter, not a convention — the orchestrator and SQL layer never receive the other ~200 permissions. A new, broad, or unexpected permission string added to the identity system later cannot silently expand data access here; it has to be explicitly added to this frozenset first.

core/rbac.py

```
DATA_VISIBILITY_PERMS: frozenset = frozenset({
    'view list of all inspections',
    'view list of own inspections',
    'view own inspection dashboard',
    'view own inspection portfolio list',
    'view list of own inspection schedules',
    'view list of own inspection mitigative actions',
    'view list of all inspection mitigative actions',
    'conduct an inspection',
    'can be an inspectee',
    'view po details in any mitigative action',
    'view inspection cycle list',
    'view project inspection dashboard',
})
```

fastapi_app.py — /api/chat

```
_user_perms = token_data['permissions']
if _user_perms:
    _data_vis = [p for p in _user_perms if p in DATA_VISIBILITY_PERMS]
    visibility_scope = {'sub': sub, 'permissions': _data_vis}
else:
    visibility_scope = None   # → hard BLOCK downstream, never silently unrestricted
```

5. RBAC scope resolution — `build_scope_filter`

This single function turns a permission list into exactly one of three outcomes: an empty string (unrestricted), a SQL WHERE-clause fragment (scoped), or the literal string `'BLOCK'`. Every caller in the system treats these three return shapes as the entire contract of this function.

`core/rbac.py::build_scope_filter`

```
if visibility_scope is None or visibility_scope == {}:
    return ''                       # ⚠ see fail-open note in §11

perms = set(visibility_scope.get('permissions', []))
if not perms:
    return 'BLOCK'

if 'view list of all inspections' in perms:
    return ''                       # unrestricted

user_subq = f"(SELECT id FROM users WHERE id_fk = {int(sub)} AND active = true AND is_synced =
true)"

is_inspector = bool(perms & _INSPECTOR_PERMS)
is_inspectee = 'can be an inspectee' in perms

if is_inspector and is_inspectee:
    return f"(ir.inspector_user_id = {user_subq} OR ir.inspectee_user_id = {user_subq})"
if is_inspector:
    return f"ir.inspector_user_id = {user_subq}"
if is_inspectee:
    return f"ir.inspectee_user_id = {user_subq}"
return 'BLOCK'
```

5.1 How role is determined

`_INSPECTOR_PERMS` deliberately contains only permissions that *exclusively* mean "this person conducts inspections":

```
_INSPECTOR_PERMS: frozenset = frozenset({
    'conduct an inspection',
    'view own inspection portfolio list',
    'view list of own inspection schedules',
})
```

Permissions like `view own inspection dashboard` are deliberately excluded from this set even though inspectors hold them, because FM Manager (inspectee) roles hold the identical permission string. Including a shared permission here would give inspectee-only accounts the wrong predicate column

(`inspector_user_id` instead of `inspectee_user_id`). **Rule for anyone adding a new permission to this set: it must be exclusive to the inspector role, never shared with inspectee-type roles.**

5.2 Role names never appear in this file

Only permission strings drive every branch above — never a role/title lookup. This matches how the identity API actually describes users (by permission, not by a single role field) and means a user can genuinely hold both an inspector permission and the inspectee permission at once, producing the dual-role OR predicate rather than one role silently winning.

6. SQL-level enforcement — `enforce_scope`

This is the layer that actually matters for data security. `enforce_scope(sql, scope_filter)` parses the LLM-generated SQL into a real syntax tree (via `sqlglot`) and injects the scope predicate directly into it. It does not trust, inspect, or care what predicate — if any — the model already wrote.

> **CONTRACT, VERBATIM FROM THE CODE**
>
> *scope_filter == '' → unrestricted, return SQL unchanged. scope_filter == 'BLOCK' → raise ValueError('SCOPE_BLOCKED'). anything else → inject into every Select that directly FROM/JOINs inspection_report, or a secondary-scoped table.*

6.1 The two shapes of query this layer has to handle

1. **Direct case:** the SQL joins `inspection_report` directly (the common case) — inject the predicate straight into that `SELECT`'s WHERE clause.
2. **Indirect case:** the SQL reaches owner-restricted data *without* going through `inspection_report` at all — e.g. `SELECT COUNT(*) FROM inspection_corrective_action`, or `SELECT * FROM users`. There is no `inspection_report` predicate to attach to, so the ownership relationship for that specific table has to be reconstructed a different way.

6.2 Every SELECT node is scoped independently

The injection walks the parsed tree and evaluates each individual `SELECT` node on its own terms — including every branch of a UNION, every CTE body, and every nested subquery. This matters because a single statement can legitimately mix a branch that joins `inspection_report` with a branch that reaches a secondary-scoped table bare; both need to be caught independently, not decided once for the whole statement.

`core/validator.py — the injection function`

```
def _inject(node):
    if not isinstance(node, exp.Select):
        return node
    if _alias_for_table(node, 'inspection_report'):
        return node.where(scope_expr.copy())
    for tbl_name, spec in secondary_tables:
        alias = _alias_for_table(node, tbl_name)
        if alias:
            node = node.where(sqlglot.parse_one(
                self._build_secondary_predicate(tbl_name, spec, alias, scope_filter, ir_predicate)
            ))
    return node

rewritten = tree.transform(_inject).sql(dialect="postgres", pretty=False)
```

6.3 The secondary-scope table registry

`_SECONDARY_SCOPE_TABLES` lists every table that carries owner-restricted data but doesn't necessarily join `inspection_report` in a given query, along with how to reach the requester's own rows from that table:

Table	Mode	How ownership is reached
`inspection_cycle`	`forward`	`inspection_report` holds the FK to this table
`inspection_corrective_action`	`reverse`	one hop: `ica.inspection_id → inspection_report.id`
`po_details` , `pr_details`	`reverse2`	two hops: `→ inspection_corrective_action → inspection_report`
`inspection_schedule`	`direct`	carries its own `inspector_id` / `inspectee_id` columns — no join path to `inspection_report` exists; the predicate's column names are rewritten to match
`inspector_portfolio`	`inspector_only`	owned via `user_id` , meaningful only for the inspector role — an inspectee-only requester gets `1 = 0` (zero rows), not an incorrect predicate
`inspector_portfolio_details`	`inspector_only_reverse`	one hop to `inspector_portfolio` , then the same inspector-only rule
`users`	`connected_users`	restricted to people who appear as inspector *or* inspectee on the requester's own inspections — not a full directory, not blocked outright either

> **FACILITY IS DELIBERATELY NOT IN THIS TABLE**
>
> Facility names are treated as public reference data, same as country/region/inspection type — every authenticated user can list facility names regardless of scope. This is an intentional design decision documented directly in the code (`_SECONDARY_SCOPE_TABLES` 's leading comment), not an oversight.

6.4 Worked example — the `reverse` mode

A restricted inspector asks a question that resolves to a bare corrective-action count, with no join to `inspection_report` at all:

AS GENERATED BY THE MODEL

```
SELECT COUNT(ica.id) AS total_corrective_actions
FROM inspection_corrective_action ica
WHERE ica.active = true;
```

AS REWRITTEN BY ENFORCE_SCOPE BEFORE EXECUTION

```
SELECT COUNT(ica.id) AS total_corrective_actions
FROM inspection_corrective_action AS ica
WHERE ica.active = TRUE
  AND ica.inspection_id IN (
      SELECT id FROM inspection_report
      WHERE inspector_user_id = (SELECT id FROM users WHERE id_fk = 11410 AND active = true AND
is_synced = true)
        AND active = true
  );
```

6.5 Worked example — the `connected_users` mode

The same account asks for the user directory:

AS GENERATED BY THE MODEL

```
SELECT u.id, u.first_name, u.last_name, u.email
FROM users u
WHERE u.active = true
ORDER BY u.last_name, u.first_name;
```

AS REWRITTEN BY ENFORCE_SCOPE

```
SELECT u.id, u.first_name, u.last_name, u.email
FROM users AS u
WHERE u.active = TRUE
  AND u.id IN (
      SELECT inspector_user_id FROM inspection_report
        WHERE inspector_user_id = (SELECT id FROM users WHERE id_fk = 11410 ...) AND active = true
      UNION
      SELECT inspectee_user_id FROM inspection_report
        WHERE inspector_user_id = (SELECT id FROM users WHERE id_fk = 11410 ...) AND active = true
  )
ORDER BY u.last_name, u.first_name;
```

6.6 Worked example — a multi-branch (UNION) query

A single statement combining a scoped branch and a bare-table branch is scoped branch-by-branch, not as one all-or-nothing decision:

```
SELECT 'My Inspections', COUNT(*) FROM inspection_report ir
  WHERE ir.inspector_user_id = (SELECT id FROM users WHERE id_fk = 11410 ...) ...
UNION ALL
SELECT 'Total Corrective Actions', COUNT(*) FROM inspection_corrective_action ica
  WHERE ica.active = TRUE;
```

Each branch is its own `SELECT` node in the parsed tree. Branch 1 is caught by the direct `inspection_report` check; branch 2 has no such join, so it falls through to the `reverse` secondary-table

predicate for `inspection_corrective_action` — the same predicate shown in §6.4 — appended to that branch specifically. The same logic applies uniformly to CTEs and nested subqueries, since a CTE body and a subquery are both just `SELECT` nodes elsewhere in the same tree.

7. How to add a new protected table

When a new table is introduced that carries owner-restricted rows (i.e. rows that belong to a specific inspector or inspectee and shouldn't be visible to everyone), it needs an entry in `_SECONDARY_SCOPE_TABLES` — otherwise any query that reaches it without an explicit `inspection_report` join in the same `SELECT` is invisible to the scope layer entirely.

1. **Determine the ownership path.** Does the new table have a direct FK to `inspection_report` (→ `reverse`)? A FK to an intermediate table that itself FKs to `inspection_report` (→ `reverse2`)? Its own `inspector_id` / `inspectee_id`-shaped columns with no join path at all (→ `direct`)? Ownership meaningful only for one role, e.g. a standing assignment table (→ `inspector_only` / an equivalent `inspectee_only` could be added symmetrically)? Or is it a lookup that needs a bespoke "connected to my own data" rule (→ a new custom mode, following the `connected_users` pattern)?
2. **Add the entry** to `_SECONDARY_SCOPE_TABLES` with the appropriate `mode` and its required fields (see §6.3 for the shape each mode expects).
3. **If no existing mode fits**, add a new branch to `_build_secondary_predicate()` that returns the WHERE-clause fragment for that table, and document the reasoning the same way the existing modes are documented — nothing about the runtime dispatch (§6.2) needs to change, since it already iterates `_SECONDARY_SCOPE_TABLES` generically.
4. **Confirm it does not belong on the public/reference-data exception list instead** (the `facility` precedent in §6.3) — not every table needs scoping; pure lookup/reference tables with no row-level owner don't.

No caller-side change is required — `enforce_scope()` discovers which secondary tables are present in a given query by checking table-name presence in the SQL text, so a new registry entry takes effect immediately for any query touching that table, in any position in the query (top-level, UNION branch, CTE, or subquery).

8. Write-operation blocking

Independent of role or scope, any generated SQL is hard-rejected before reaching the database if it matches:

core/validator.py

```
_DML_RE = re.compile(r"\b(INSERT|UPDATE|DELETE|DROP|ALTER|TRUNCATE)\b", re.IGNORECASE)

if self._DML_RE.search(sql):
    issue = ValidationIssue("HARD_FAIL", "DML_BLOCKED",
        "Only SELECT queries are allowed — DML/DDL detected", retryable=False)
    return ValidationResult(passed=False, issues=[issue])
```

This check runs before scope injection and is non-retryable — the model gets no second attempt to phrase the same mutation differently within the same turn. This is the whole system's write boundary: there is currently no code path, for any role, that executes anything other than a `SELECT`.

9. Information-disclosure redaction

The system prompt instructs the model never to reveal the scope predicate or the internal user-ID mapping to the end user. Because this is a prompt instruction rather than a code-level guard, it is not reliable on its own — the model can be asked directly or indirectly about its own mechanism and may comply. A deterministic backstop is therefore applied to every response, independent of what the model decided to say.

`fastapi_app.py`

```
_SCOPE_PREDICATE_RE = re.compile(
    r"ir\.(?:inspector|inspectee)_user_id\s*=\s*\(\s*SELECT\s+id\s+FROM\s+users"
    r"\s+WHERE\s+id_fk\s*=\s*\d+[^)]*\)", re.IGNORECASE)
_ID_FK_RE      = re.compile(r"id_fk\s*=\s*\d+", re.IGNORECASE)
_DRUPAL_UID_RE = re.compile(r"Drupal\s+UID[\s:(]*\d+\)?", re.IGNORECASE)

def _redact_security_internals(text):
    text = _SCOPE_PREDICATE_RE.sub("[access-control filter]", text)
    text = _ID_FK_RE.sub("id_fk = [redacted]", text)
    text = _DRUPAL_UID_RE.sub("[redacted]", text)
    return text
```

Applied to both the answer text and the model's reasoning trace, at the single point right before the HTTP response is built — the same boundary-guard pattern used to strip raw UUID columns from result tables (`_strip_uuid_columns`, same file). This redacts the literal SQL predicate and any mention of the internal Drupal UID, in whatever surrounding punctuation the model happened to use.

> **DESIGN NOTE**
>
> This regex targets SQL-shaped and UID-shaped text specifically. A model that describes the same mechanism in pure prose without those literal patterns (e.g. "I only show data tied to your account record") would not be caught by this layer — the underlying data access is unaffected either way, since this control only concerns what the model *says about* the mechanism, not what data it can reach.

10. Security audit logging

A dedicated logger, `inspectai.security`, emits one structured line per scope decision, tagged `[SCOPE]` for filtering. It records three moments in a request's lifecycle:

- **Pre-decision:** the raw permission list fetched for this request (`fastapi_app.py`)
- **Decision:** `outcome=UNRESTRICTED|SCOPED|BLOCKED`, the sub, permission count, the resolved filter, and the question (`agent/orchestrator.py`)
- **Enforcement:** which table(s) actually triggered injection for this specific query — e.g. `via=['users']` or `via=['inspection_report', 'po_details']` (`core/validator.py`)

Example — a query touching three tables in one statement

```
2026-07-10 16:05:06 | INFO | [SCOPE] qid=c420a31f outcome=SCOPED sub=11410 perms=9
  filter='ir.inspector_user_id = (SELECT id FROM users WHERE id_fk = 11410 ...)'
  question='give me a two-row table: ...'
2026-07-10 16:05:24 | INFO | [SCOPE] injecting scope filter:
  via=['inspection_report', 'inspection_corrective_action', 'users']
  filter=ir.inspector_user_id = (SELECT id FROM users WHERE id_fk = 11410 ...)
```

This logger shares the same rotating log file as the rest of the application (`debug_llm_YYYYMMDD_HHMMSS.log`, one file per server start) rather than a dedicated security-only file, so a full day's activity stays in one place while still being trivially filterable: `grep '\[SCOPE\]' debug_llm_*.log`.

11. Design limitations & things to revisit

1. **Prose-paraphrased disclosure is not caught.** The redaction in §9 catches SQL-shaped and UID-shaped leaks specifically. A model that explains the mechanism in pure prose without those patterns would not be redacted by this layer.
2. **Scope injection on a LEFT JOIN forces effective inner-join behavior.** Appending the scope predicate to a WHERE clause on a query that LEFT-joins a scoped table (e.g. `users` LEFT-joined to `inspection_report`) causes any row where the joined table's ownership column is NULL to be filtered out, same as an INNER JOIN would. This is the safe direction for security (excludes rather than leaks), but would silently drop legitimate zero-count rows for a feature that wants to preserve LEFT JOIN semantics (e.g. "show all facilities, including ones with zero of my inspections").
3. **The `users` table's `connected_users` restriction is a least-privilege design choice, not a directly specified business requirement.** It limits visibility to people connected to the requester's own inspections rather than blocking the table outright or leaving it fully open. Worth confirming this matches actual product intent if the requirement is ever formalized.
4. **The string-fallback injection path (used only if `sqlglot` is unavailable) cannot correctly handle multi-branch queries.** It performs a single regex-based text insertion and has no per-branch awareness. Not currently exercised in practice — `sqlglot` is a hard dependency and the AST path in §6 is what actually runs — but this would need attention if that dependency were ever made optional.
5. **Fail-open contract on `build_scope_filter`.** Documented directly in the function's own docstring: passing `{}` instead of `None` for an "unfetched permissions" state returns unrestricted access, not a block. The current caller in `agent/orchestrator.py::query()` converts an unfetched state to `None` before calling this function, so the trap is inert today — but any new caller of `build_scope_filter` must follow the same `None`-not-`{}` contract, or it will silently grant full access.
